比较 C++ 中 for 循环与递归计算斐波那契数列的耗时

结论：**for循环求和占用的时间远少于函数递归**，核心原因是递归存在大量重复计算和函数调用开销，具体差异如下：

1. 时间差异的核心原因

（1）for循环（迭代方式）

- 时间复杂度：O(n)，仅需从第1项/第2项开始，依次计算到第n项，**每一项只计算1次**，无重复操作。
- 额外开销：几乎没有多余开销，仅需一个循环变量和几个临时变量，内存占用固定，执行效率极高。

（2）函数递归（递归方式）

- 时间复杂度：$O(2^n)$，存在严重的**重复计算**（比如计算f(5)需要f(4)和f(3)，计算f(4)又需要f(3)和f(2)，f(3)被重复计算2次，n越大，重复次数呈指数级增长）。
- 额外开销：每次递归都会创建函数栈帧（保存返回地址、参数、局部变量），频繁的函数调用和栈帧销毁会消耗大量时间和内存，n较大时甚至会出现栈溢出。

2. 补充说明（结合C++指针/代码场景）

- 即使优化递归（比如用“记忆化递归”，将已计算的项存入数组/指针指向的内存空间，避免重复计算），其时间复杂度能降到O(n)，但仍不如for循环高效（仍有函数调用开销）。
- 实际开发中，计算斐波那契数列第n项（n较大时），优先用for循环；递归仅适用于n极小（如n≤10）的场景，或用于理解递归思想。

需要我帮你写一段**C++代码**，对比for循环和递归计算斐波那契数列的耗时，更直观看到差异吗？

（注：文档部分内容可能由 AI 生成）